

Splendor as a Fast Reinforcement Learning Environment: League Self-Play, Elo Evaluation and Search

Aditya Kumar

Carnegie Mellon University `adityaku@andrew.cmu.edu`

September 2026

Abstract

We present an exact-rules implementation of the board game *Splendor* (2–4 players) as a reinforcement learning environment for PufferLib. The game logic is a single C header that advances roughly four million turns per second on one CPU core, exposes a legal-action mask inside the observation, and supports state snapshots for tree search. On top of it we build a prioritized fictitious self-play league for PPO, a Bradley–Terry Elo ladder with fixed heuristic yardsticks, a batched PUCT Monte Carlo tree search with an AlphaZero-style training loop, and a browser interface for watching or playing against agents. A 300M-step league run on a single workstation produces a 2-player policy rated about 230 Elo above a strong greedy heuristic and 215 Elo above plain self-play with the same network and budget; 64 simulations of search add a further 100 Elo. Code, trained models and the interface are released under the MIT license at <https://github.com/Sylindril/splendor>.

1 Introduction

Splendor is a resource-management card game with hidden information (deck order and face-down reserved cards), chance (card draws), sub-turn decisions (returning tokens above the ten-token limit, choosing among nobles) and two to four players. These properties make it a useful mid-sized testbed between fully observable two-player board games and large multi-agent simulations. Existing implementations are written for human play or for slow Python search and are not suitable for training policies with hundreds of millions of samples.

This note describes a from-scratch environment designed for throughput and simplicity, and the training and evaluation tooling around it. The contributions are practical: (i) an exact-rules engine with a compact fixed-size observation and a 72-way discrete action space that is verified by sanitizer runs and invariant tests; (ii) a league training wrapper that removes idle transitions and mixes the current policy with a pool of past checkpoints sampled by prioritized fictitious self-play (PFSP) [2]; (iii) an Elo ladder built on Bradley–Terry ratings [3] with fixed bots as anchors; (iv) a batched PUCT search that uses engine snapshots and determinization, and (v) shipped models with an interface.

2 Environment

Rules. The engine implements the published rules without simplification: 90 development cards in three tiers with four face-up per tier, ten nobles of which $P + 1$ are dealt, a bank of 4/5/7 gems per color for 2/3/4 players plus five gold jokers, the four turn actions (take up to three different

gems, take two of one color when at least four remain, reserve a face-up or deck-top card gaining a gold, buy a card paying with bonuses then gems then gold), the ten-token limit enforced by a discard phase, noble visits after a purchase with a choice when several qualify, and game end at the end of the round in which a player reaches 15 points, with the fewest-cards tie-break. The start seat is randomized per game so that seat index carries no advantage. The card table was cross-checked against four independent open-source implementations.

Interface. One C struct holds one game; each seat is a PufferLib agent, so a vectorized environment of N games exposes NP agents. One environment step is one turn of the seat to move; other seats’ actions are ignored and their observations carry a pass-only mask. Sub-phases (discard, noble choice) are additional steps by the same seat. The observation is a fixed vector of $240 + 48P$ bytes: bank, deck sizes, twelve face-up cards (cost, bonus, points), five noble requirement vectors, per-player blocks (tokens, bonuses, points, three reserved cards with a presence flag; opponents’ deck-top reserves are hidden), the 72-way legal-action mask, the turn number and a to-move flag. Policies read the mask from the observation and mask their logits, so no invalid action is ever sampled and the trainer needs no masking support. The engine additionally provides opaque state snapshots, restore, a no-op action that freezes a game, and a *determinize* operation that re-deals everything a given seat cannot see.

Throughput. On an Apple M4 Max, a single core advances 4.1M turns/s with random actions and 1.9M turns/s when actions are sampled uniformly from the mask in NumPy. Address and undefined-behavior sanitizers over one million random steps at each player count report no findings; invariant tests check token, card and noble conservation, the ten-token limit, and that illegal actions are no-ops.

3 Training

Policy. A three-layer, 512-unit LayerNorm MLP over the scaled observation with masked actor and value heads (0.74M parameters). Plain self-play, in which every seat is the current policy, works but wastes the idle seats’ transitions and can cycle.

League. A wrapper exposes only the learner’s seat to the trainer. Inside one wrapper step it applies the learner’s action, then plays every opponent seat (including their sub-phases) with frozen policies evaluated on CPU inside the worker process, batched per model, until the learner is to move again. Every trainer sample is therefore a real decision. Opponents are assigned per game: with probability $\frac{1}{2}$ the latest weights (reloaded from disk every few seconds), otherwise a member of a pool of past checkpoints sampled with PFSP weights $(1 - w_i)^2 + 0.05$, where w_i is an exponential moving average of the learner’s win rate against member i . A checkpoint joins the pool every 20M steps. The PPO implementation is PufferLib’s [1], with V-trace corrections and prioritized minibatches, reward ± 1 for win/loss plus 0.02 per prestige point. The wrapper sustains about 180K learner steps per second per process on the reference machine; end-to-end training runs at 150–180K samples per second with eight workers and an MPS device.

4 Evaluation

Ladder. Ratings are fitted with the Bradley–Terry model by minorization–maximization on pairwise results (draws count half), with a one-game prior so that undefeated participants remain finite,

reported on the Elo scale ($400 \log_{10}$) and anchored at a uniform-random-legal agent. Two-player ladders are round robins; three- and four-player ladders sample random tables and extract pairwise results from final ranks. A fixed greedy heuristic (buy the most valuable affordable card, else take gems toward the closest card) serves as a stable yardstick: it beats random in 100% of games.

Participant (2 players, 100 games per pair)	Elo	Win %
League, 300M steps, final	1210	75.7
League pool, 240M	1197	74.5
League pool, 160M	1049	58.0
Plain self-play, same network, 60M	994	52.4
Greedy heuristic	982	51.0
League pool, 80M	945	46.8
League pool, 40M	740	28.9
Random	0	6.7

Table 1: Two-player ladder. The league policy gains about 230 Elo over the greedy bot and 215 over plain self-play; the curve is flattening but still rising at 300M steps.

Participant (4 players, 150 random tables)	Elo
League, 200M steps, final	1017
League pool, 175M	1009
League pool, 125M	971
Greedy heuristic	894
League pool, 75M	886
League pool, 25M	699
Random	0

Table 2: Four-player ladder after a shorter run; the margin over the heuristic is smaller (120 Elo) with a third of the per-seat experience and higher outcome variance.

5 Search

The search is a batched PUCT over engine snapshots: per simulation round every root descends to a leaf in lock-step, one vectorized environment step expands all leaves, and one network forward pass evaluates them. Values are per-seat vectors (the network is evaluated on every seat’s observation of the leaf), so multiplayer backups are exact; terminal leaves use the engine’s rewards. Hidden information is handled by determinizing the root for the seat to move, and the engine’s random state is part of the snapshot, so a search explores one determinization. With the 512-unit network the search runs about 26K simulations per second on CPU at batch 64. Sixty-four simulations per move beat the raw policy they search with in 63% of games, which the ladder reports as roughly 100 Elo. An AlphaZero-style loop (self-play with search, then cross-entropy on visit distributions plus value regression) is included and warm-starts from a PPO checkpoint; scaling it is left to future work.

6 Interface

A dependency-free browser interface renders a square table with a mat per player, animates every move from a structured event emitted by the server (cards flying to mats, replacement cards flipping in from the deck, gems moving to and from the bank), holds the finished board for inspection with standings, supports keyboard control, and lets the user seat any mix of humans, bots, checkpoints and search agents.

7 Limitations

No human rating is available, so “superhuman” claims cannot be made; the ladder measures strength relative to bots and past checkpoints only. Rewards include small prestige-point shaping. The search uses a single determinization per move. Runs used one workstation; the league is designed to scale to many workers on a GPU host but this was not benchmarked here.

8 Conclusion

A small, exact and fast environment plus a league wrapper gives a clear and cheap path from random play to a policy well beyond a strong heuristic in under an hour on a desktop, with search as a further multiplier. We hope the code and models are useful as a reproducible mid-sized benchmark for multi-agent reinforcement learning.

References

- [1] J. Suarez. PufferLib: Making reinforcement learning libraries and environments play nice. *arXiv:2406.09411*, 2024.
- [2] O. Vinyals et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575:350–354, 2019.
- [3] D. R. Hunter. MM algorithms for generalized Bradley–Terry models. *Annals of Statistics*, 32(1):384–406, 2004.
- [4] D. Silver et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419):1140–1144, 2018.